**Name:** Nada Elsayed Mohamed Ahmed Elsayad **Matrícula num.:** 256292
**Professor Name:** Roveri Marco
**Subject Name:** Automated Planning: Theory and Practice [145862]

---

# Interplanetary Museum Vault Project Report

## 1. Project Overview

This project models the Interplanetary Museum Vault (IMV) scenario across five planning problems, starting from a simple classical PDDL formulation and progressing to HTN, temporal planning, and finally PlanSys2 execution.The core IMV environment is defined by:

- The environment contains the locations: entrance, tunnel, hallA, hallB, cryo, pod1, pod2, and stasis.
- The robot must move artifacts between these locations according to the museum constraints.
- Artifacts are initially distributed as: a1, a2 in hallA, b1, b2 in hallB and cs1, cs2 in cryo.
- The common mission goal is: move a1 and a2 to cryo, move b1 and b2 to anti-vibration pods( pod1 and pod2), and move cs1 and cs2 to stasis.

The project development directly follows the step-by-step progression outlined in the main.pdf assignment, which mandates the development of the same overall museum scenario using different planning paradigms:

- **Problem 1:** classical baseline
- **Problem 2:** capacity and multi-agent extension
- **Problem 3:** HTN/HDDL version of the same scenario
- **Problem 4:** temporal / durative version
- **Problem 5:** PlanSys2 implementation using fake actions

The whole project implementation and results exists in this Git repository https://github.com/NadaElsayadUni/AutomatedPlaningAndTheory.

---

## 2. Model Ontology and Design

Some assumptions were used to keep the models solvable and easy to explain:

- The museum map is abstracted into symbolic connected locations instead of geometric coordinates.
- Artifacts are indivisible objects; only their location matters.
- Robot carrying capacity is modeled with slots.
- Only constraints that materially affect planning are modeled explicitly.
- More detailed realism is introduced gradually across problems instead of all at once in 1st Problem.

### Predicates used:

The main predicates were chosen because they directly represent the planning state. Each predicate has a specific role in controlling where the robot is, where the artifacts are, what the robot can carry, and which actions are legally allowed as:

- robot_at / at: robot location, the core way to track where the transport agent currently is.
- artifact_at: artifact location, records where each artifact is currently stored.
- robot_carrying, empty: single-carry abstraction in Problem 1. empty tells whether the robot is free to pick something up, and robot_carrying records which artifact is currently being transported.

- slot_free, in_slot, robot_slot: explicit capacity representation in Problems 2–5, replacing the simpler empty, robot_carrying abstraction once capacity becomes important. It states which slots belong to which robot, tracks whether a given slot is available, and records which artifact is stored in which slot, respectively.
- connected: map connectivity, represents the layout of the museum as a graph.
- manip_free: resource availability for the robot that prevents impossible overlap between actions in temporal domains. By requiring manip_free at the start of an action and restoring it at the end, the model enforces that one robot can only perform one durative action at a time.
- tunnel, non_tunnel: distinguish tunnel logic from normal rooms. It is useful because tunnel traversal follows different rules from normal movement without hardcoding.
- sealing_on, sealing_off: represent vault sealing state for safe tunnel traversal.
- hall_b_safe: represent seismic accessibility of Hall Beta in the seismic domain which allows the model to restrict all Hall Beta actions to safe windows only.

These predicates were kept only when they were needed by the corresponding problem. For example, manip_free and sealing predicates were not introduced until the temporal model, because they are not necessary in the simpler classical versions.

---

# 3. Implementation and Problem Progression

Subsequent problems introduce additional complexities, such as adding capacity, multiple agents, HTN decomposition, durative actions, tunnel sealing, seismic access constraints, and PlanSys2 execution; a more detailed explanation of each of these challenges is provided below.

## 3.1. Problem 1: Classical PDDL Baseline

Problem 1 asks for a classical PDDL model of the IMV scenario with one robotic curator that can carry one artifact at a time. This problem acts as the baseline classical formulation. The goal was to first capture the core transport logic before adding more realism in the later problems. For that reason, the model is intentionally simple and focuses on reachability, carrying, and final artifact placement.

### The baseline is structured around:

- one robot r1
- one-artifact carrying capacity using empty and robot_carrying
- actions:
  - move
  - load
  - unload
- artifact goal structure:
  - a1, a2 to cryo
  - b1, b2 to pod1, pod2
  - cs1, cs2 to stasis
- no explicit seismic timing for Hall Beta
- no explicit tunnel sealing
- no explicit cooling or temperature threshold
- one tunnel node instead of a more detailed path structure

### Files:

- problem1/domain.pddl:
  - Defines the classical domain “imv”.

- Introduces the base predicates robot_at, artifact_at, robot_carrying, empty, and connected.
- Keeps the action set intentionally minimal with only move, load, and unload.

- problem1/problem.pddl:
  - Defines the initial museum map and objects.
  - Place the robot at the entrance.
  - Place each artifact in its starting room.
  - Encodes the final placement goals for all six artifacts.

### Results

The problem was fully modeled and solved with more than one classical planner. The generated plans confirm that the baseline domain is valid and that the target goal configuration is reachable.

**Files:** problem1/results/ff.plan, problem1/results/lama.plan, problem1/results/lama_first.plan, problem1/results/optic.plan

**Observed results:**

- FF, LAMA, and LAMA-first all produced the same basic plan structure with 32 actions.
  - The classical planners start with Hall Alpha and Cryo / Stasis work, then finish Hall Beta
- OPTIC also solved the problem and produced a 32-action plan, but with a different ordering
  - OPTIC starts from Hall Beta first, then continues with Hall Alpha and Cryo / Stasis
- Despite the different order, the solution quality is essentially the same because the model is still classical and unit-cost.
- Also, OPTIC is not worse, but it does not give a meaningful advantage to this non-temporal model.
- For more details, please check the *"Appendix 6.1 section"*.

---

## 3.2. Problem 2: Capacity and Multi-Agent Extension

Problem 2 extends the same museum scenario by adding explicit carrying capacity and, optionally, a second transport agent. The purpose of this step was to move from a simple single-item courier model to a richer formulation where planner quality can improve by using multiple slots or a second robot. It still keeps some early abstractions like; no explicit sealing, seismic time windows and temperature or cooling constraints. The focus here is specifically on capacity and agent structure, not yet on temporal realism.
The key adjustment was replacing the binary "carrying or empty" model with a structural slot model. This makes the domain more expressive and lets the planner carry two artifacts at once when useful. I also added a multi-agent version to explore whether cooperation improves the solution.

**The problem is structured around:**

- slot-based capacity instead of single empty state
- one curator with two slots in problem.pddl, domain.pddl
- curator + drone without fixed roles problem-curator-drone.pddl
- curator + drone with forced assignment domain-assigned.pddl

### Files:

- problem2/domain.pddl
  - main classical domain with capacity and multi-agent support
  - introduces slot_free, in_slot, and robot_slot
  - move, load, and unload now take robot and slot parameters
- problem2/problem.pddl

    - single-curator version of Problem 2
    - r1 has two slots (slot1, slot2)
    - same museum layout and same final artifact goals as Problem 1
- problem2/problem-curator-drone.pddl
    - adds a second robot (drone) d1
    - curator and drone both operate on the same map
    - used to test whether the planner can benefit from multi-agent transport
- problem2/domain-assigned.pddl
    - extends the main domain with predicate handles
    - forces artifacts to be associated with specific robots
- problem2/problem-curator-drone-assigned.pddl
    - uses domain-assigned.pddl
    - fixes the division of labour between r1 and d1, so forcing the cooperation between two agents

### Result

Both the single-curator and multi-agent variants were tested.

**Files:** problem2/results/ff.plan, problem2/results/lama.plan, problem2/results/problem-curator-drone.pddl.plan, problem2/results/problem-curator-drone-assigned.pddl.plan

**Observed result:**

- The single-curator two-slot model gives 24 actions, which is better than Problem 1's 32 actions.
    - This is due to having two artifacts that can be transported in one trip.
- In the unassigned curator–drone model, some planners use only d1, so adding another agent does not automatically improve the plan.
- LAMA curator–drone plans vary between 28, 27, and 24 actions
- The assigned curator–drone plans use both robots, but are about 30 actions.
- Multi-agent planning is only clearly beneficial when the role structure encourages or forces cooperation.
- The assigned domain is not the shortest in action count, but it is the clearest demonstration of division of labour.
- For more details, please check the *"Appendix 6.2 section"*.

---

## 3.3. Problem 3: HTN / HDDL Model

Problem 3 keeps the same basic museum scenario but changes the modeling paradigm from classical planning to HTN/HDDL. The goal is no longer expressed only as a final state to satisfy; instead, it is decomposed through tasks and methods into an explicit hierarchy of deliveries. Problem 3 does not change the primitive transport logic very much. This makes Problem 3 useful for comparing not just plan quality, but also the effect of decomposition design.

**This problem is structured around:**

- the same primitive transport actions as the Problem 2 baseline
- one top-level compound task
- one delivery subtask per artifact objective
- methods that decompose high-level tasks into primitive steps

**Files**

- problem3/domain.hddl
  - main HTN domain
  - keeps the same primitive actions as the Problem 2 baseline
  - adds compound tasks such as relocate_all, deliver_a1_to_cryo, and similar named deliveries
  - defines methods that decompose those tasks into action sequences
- problem3/problem.hddl
  - HTN problem file
  - keeps the same museum objects and initial state as the single-curator Problem 2 baseline
  - encodes the mission through the top-level task (relocate_all)
- problem3/domain-pair.hddl
  - The paired HTN variant was introduced to test whether a better decomposition design can reduce unnecessary travel compared with the more direct one-delivery-at-a-time decomposition.

## Results

**Files**: problem3/results/panda.txt, problem3/results/panda-pair.txt

**Observed result:**

- panda.txt gives a long decomposition with about **46 primitive actions.**
- panda-pair.txt gives a much shorter decomposition with about **30 primitive actions.**
- The quality of the HTN solution depends strongly on the method design.
- The paired HTN version is better because it groups work and uses the slot structure more effectively.
- This shows that in HTN planning, the decomposition itself is part of the solution quality.
- For more details, please check the *"Appendix 6.3 section"*.

---

## 3.4. Problem 4: Temporal / Durative Planning

Problem 4 converts the same museum mission into a temporal model. It extends the earlier abstraction by explicitly representing: the maintenance tunnel as a constrained transition, sealing as its own durative process, concurrency only where it is realistic, seismic safety windows for Hall Beta in the seismic variant.

**This problem is structured around:**

- durative actions instead of instantaneous actions,
- explicit sealing logic for tunnel traversal,
- resource-style control through manip_free,
- optional multi-agent parallelism,
- and a seismic variant with timed Hall Beta constraints.

**Files**

- problem4/domain.pddl
  - main temporal domain
  - introduces durative versions of movement, loading, unloading, and sealing
  - distinguishes tunnel vs non-tunnel locations
- problem4/problem.pddl

    - main single-robot temporal problem
    - includes sealing state and tunnel typing in the initial state
    - uses (:metric minimize (total-time))
- problem4/problem-curator-drone.pddl
    - adds a second robot for temporal parallelism tests
- problem4/domain-seismic.pddl
    - seismic temporal domain
    - adds hall_b_safe
    - uses Hall Beta-specific actions to make the seismic logic explicit
- problem4/problem-seismic.pddl
    - single-robot seismic temporal problem
    - uses timed initial literals for Hall Beta safe/unsafe windows
- problem4/problem-curator-drone-seismic.pddl
    - multi-agent seismic temporal problem
- problem4/scripts/respace_plan_val.py
    - post-processing utility used to make temporal plans easier for VAL to validate

### Durations used

The durations express relative effort rather than physical seconds, which was enough to create meaningful temporal trade-offs without overcomplicating the model:

- Normal room-to-room movement is cheaper than tunnel traversal, move = 2.
- Tunnel traversal is longer because it needs controlled sealing, move_into_tunnel / move_out_of_tunnel = 3.
- Sealing and manipulation are short support actions, activate_sealing / deactivate_sealing = 1.
- Loading and unloading the artifacts are short, load / unload = 1.

The temporal models use standard PDDL2.1 semantics:

- at start for preconditions that must hold before the action begins.
- over all for safety conditions that must remain true for the entire action.
- at end for facts that only become true after completion.
- This is especially important for: manip_free, tunnel sealing and Hall Beta safety in the seismic domain.

## Results

**Files:** problem4/results/optic-seismic-raw.plan, problem4/results/optic-seismic-val.plan, problem4/results/optic-seismic-drone.plan, problem4/results/optic-seismic-drone-val.plan

**Observed result:**

- optic-seismic-raw.plan and optic-seismic-val.plan are the same logical plan, but the second has slightly shifted timestamps for easier VAL validation.
- The single-robot seismic plan contains about **50 durative actions.**
- The curator–drone temporal plan shows overlap between robots, but the schedule is not automatically shorter.
- OPTIC is the most useful practical planner for the seismic temporal domain.
- The multi-agent temporal plan is useful as evidence of concurrency, even when it is not the shortest one.
- For more details, please check the *"Appendix 6.4 section"*.

## 3.5. Problem 5: PlanSys2 Deployment

Problem 5 takes the planning model into execution by deploying it inside PlanSys2 with fake actions. This step is different from the previous ones because success is not only a planner finding a plan, but also the ROS 2 planning system executing the plan successfully. The most reliable workflow inside PlanSys2 was not one long monolithic final run. Instead, I used cumulative goal stages written directly into launch/imv_commands.txt. This made the execution sequence: easier to debug, more stable in practice and still equivalent to reaching the full final mission goal.

**This problem is structured around:**

- A PlanSys2 package.
- One fake action executor node per planning action.
- A temporal seismic domain adapted for PlanSys2.
- A runtime command file that loads the problem and executes the mission.

### Files

- problem5/pddl/domain.pddl
  - the deployed temporal/seismic domain used by PlanSys2.
  - contains split actions for Hall Beta and other sites.
- problem5/launch/imv_plansys2_launch.py
  - launches PlanSys2 and all fake action executors.
- problem5/launch/imv_commands.txt
  - contains the full tested command sequence: initial problem setup, cumulative goal phases and final full goal.
- problem5/src/*.cpp
  - fake action executor nodes.
  - each node subclasses ActionExecutorClient and simulates progress for one domain action.

### Results

**Files:** problem5/results/result-seismic.txt, problem5/results/result-domain.txt

**Observed result:**

- The difference between the two files is with/without seismic logic.
- The mission succeeds in cumulative stages.
- Each stage ends with Plan Succeeded.
- The final full mission is achieved through stable staged execution inside PlanSys2.
- The staged workflow is the practical improvement that made the deployment reliable.
- For more details, please check the *"Appendix 6.5 section"*.

# 4. Comparative Analysis of Planning Paradigms

- **Capacity Optimization:**
  - Moving from the single-slot model (Problem 1) plan of *32* actions to the two-slot model (Problem 2) plan of *24* actions illustrates a 25% reduction in plan length via slot optimization.
- **Multi-Agent Trade-offs:**
  - The unassigned multi-agent model showed that adding agents does not automatically improve the plan, as some planners used only one agent, while the assigned model demonstrated clearer cooperation (even if it was slightly longer at 30 actions).
- **Hierarchical Design Quality:**
  - The naive HTN model yielded a *46*-action plan, while the paired HTN variant was significantly better at *30* primitive actions, showing that decomposition design dictates HTN solution quality.
- **Temporal Execution Strategy:**
  - Integrating temporal reasoning, action durations, tunnel sealing, and seismic safety requirements. OPTIC is the most practical planner for the temporal seismic model; validation-friendly timing is sometimes a separate post-processing step.
- **Real-World Deployment:**
  - Finally, deploying the fully temporal and seismic model into PlanSys2 using executable fake actions to simulate real-world execution.

---

# 5. Conclusion

This project demonstrates a complete progression in automated planning, evolving from a classical symbolic formulation to a temporal, execution-aware model deployed within the PlanSys2 framework. The core methodology followed a controlled, incremental increase in problem realism and complexity.

We began with a basic transport task. Classical planners behaved largely equivalently at this stage: while the ordering of actions could vary, overall plan quality remained similar. We then introduced essential features such as capacity constraints and multiple agents. Moving to a two-slot capacity model produced the first major improvement in plan quality. Adding a second agent, however, only improved performance when the model either enforced cooperation or meaningfully differentiated agent roles.

Next, we restructured the problem using Hierarchical Task Networks for greater expressiveness. HTN plan quality proved to be highly sensitive to the decomposition design: the paired HTN was clearly superior to the naive, purely sequential decomposition. We then integrated temporal reasoning, action durations, tunnel sealing, and seismic safety requirements. OPTIC proved to be the most practical planner for the temporal seismic model, though obtaining validation-friendly timings sometimes required an additional post-processing step.

Finally, we deployed the fully temporal and seismic model into PlanSys2 using executable fake actions to simulate real-world execution. In practice, the most significant improvement did not come from switching planners, but from designing a better execution workflow inside PlanSys2.

This staged development process, in which each problem iteration extended and refined the previous one, produced a robust and coherent solution. It not only delivered a functional final model, but also provided a clear, demonstrable pathway for translating complex operational requirements into deployable, planning-based systems.

# 6. Appendix

This section is based on the saved result files inside the repository for each problem.

## 6.1 Problem 1: classical baseline

**Observed result:**

- FF (Fig1), LAMA, and LAMA-first(Fig2) all produced the same basic plan structure with **32 actions**
- OPTIC(Fig3) also solved the problem and produced a **32-action** plan, but with a different ordering
- the classical planners start with Hall Alpha and Cryo / Stasis work, then finish Hall Beta
- OPTIC starts from Hall Beta first, then continues with Hall Alpha and Cryo / Stasis.

```
( MOVE ENTRANCE TUNNEL )
( MOVE TUNNEL HALLA )
( LOAD R1 A1 HALLA )
( MOVE HALLA TUNNEL )
( MOVE TUNNEL CRYO )
( UNLOAD R1 A1 CRYO )
( LOAD R1 CS1 CRYO )
( MOVE CRYO TUNNEL )
( MOVE TUNNEL STASIS )
( UNLOAD R1 CS1 STASIS )
( MOVE STASIS TUNNEL )
( MOVE TUNNEL HALLA )
( LOAD R1 A2 HALLA )
( MOVE HALLA TUNNEL )
( MOVE TUNNEL CRYO )
( UNLOAD R1 A2 CRYO )
( LOAD R1 CS2 CRYO )
( MOVE CRYO TUNNEL )
( MOVE TUNNEL STASIS )
( UNLOAD R1 CS2 STASIS )
( MOVE STASIS TUNNEL )
( MOVE TUNNEL HALLB )
( LOAD R1 B1 HALLB )
( MOVE HALLB TUNNEL )
( MOVE TUNNEL POD1 )
( UNLOAD R1 B1 POD1 )
( MOVE POD1 TUNNEL )
( MOVE TUNNEL HALLB )
( LOAD R1 B2 HALLB )
( MOVE HALLB TUNNEL )
( MOVE TUNNEL POD2 )
( UNLOAD R1 B2 POD2 )
```

Fig1. FF.plan

```
(move entrance tunnel)
(move tunnel halla)
(load r1 a1 halla)
(move halla tunnel)
(move tunnel cryo)
(unload r1 a1 cryo)
(load r1 cs1 cryo)
(move cryo tunnel)
(move tunnel stasis)
(unload r1 cs1 stasis)
(move stasis tunnel)
(move tunnel halla)
(load r1 a2 halla)
(move halla tunnel)
(move tunnel cryo)
(unload r1 a2 cryo)
(load r1 cs2 cryo)
(move cryo tunnel)
(move tunnel stasis)
(unload r1 cs2 stasis)
(move stasis tunnel)
(move tunnel hallb)
(load r1 b1 hallb)
(move hallb tunnel)
(move tunnel pod1)
(unload r1 b1 pod1)
(move pod1 tunnel)
(move tunnel hallb)
(load r1 b2 hallb)
(move hallb tunnel)
(move tunnel pod2)
(unload r1 b2 pod2)
; cost = 32 (unit cost)
```

Fig2. LAMA.plan

```
; Plan found with metric 0.031
; States evaluated so far: 77
; States pruned based on pre-heuristic cost lower bound: 0
; Time 0.30
0.000: (move entrance tunnel)  [0.001]
0.001: (move tunnel hallb)  [0.001]
0.002: (load r1 b2 hallb)  [0.001]
0.003: (move hallb tunnel)  [0.001]
0.004: (move tunnel pod2)  [0.001]
0.005: (unload r1 b2 pod2)  [0.001]
0.006: (move pod2 tunnel)  [0.001]
0.007: (move tunnel hallb)  [0.001]
0.008: (load r1 b1 hallb)  [0.001]
0.009: (move hallb tunnel)  [0.001]
0.010: (move tunnel pod1)  [0.001]
0.011: (unload r1 b1 pod1)  [0.001]
0.012: (move pod1 tunnel)  [0.001]
0.013: (move tunnel halla)  [0.001]
0.014: (load r1 a2 halla)  [0.001]
0.015: (move halla tunnel)  [0.001]
0.016: (move tunnel cryo)  [0.001]
0.017: (unload r1 a2 cryo)  [0.001]
0.018: (load r1 cs2 cryo)  [0.001]
0.019: (move cryo tunnel)  [0.001]
0.020: (move tunnel stasis)  [0.001]
0.021: (unload r1 cs2 stasis)  [0.001]
0.022: (move stasis tunnel)  [0.001]
0.023: (move tunnel halla)  [0.001]
0.024: (load r1 a1 halla)  [0.001]
0.025: (move halla tunnel)  [0.001]
0.026: (move tunnel cryo)  [0.001]
0.027: (unload r1 a1 cryo)  [0.001]
0.028: (load r1 cs1 cryo)  [0.001]
0.029: (move cryo tunnel)  [0.001]
0.030: (move tunnel stasis)  [0.001]
0.031: (unload r1 cs1 stasis)  [0.001]

; * All goal deadlines now no later than 0.031
```

Fig3. OPTIC.plan

## 6.2 Problem 2: capacity and multi-agent

**Observed result for the single-curator capacity model:**

- the robot now has **two slots**
- it can load two artifacts before leaving a room
- this removes repeated travel, especially in Hall Alpha and Cryo / Stasis transport
- FF(Fig4), LAMA(Fig5), and LAMA-first(Fig6) all produce a **24-action** plan
- It's an improvement compared with Problem1's 32 actions.

```
( MOVE R1 ENTRANCE TUNNEL )
( MOVE R1 TUNNEL HALLA )
( LOAD R1 A1 HALLA SLOT2 )
( LOAD R1 A2 HALLA SLOT1 )
( MOVE R1 HALLA TUNNEL )
( MOVE R1 TUNNEL CRYO )
( UNLOAD R1 A1 CRYO SLOT2 )
( UNLOAD R1 A2 CRYO SLOT1 )
( LOAD R1 CS1 CRYO SLOT2 )
( LOAD R1 CS2 CRYO SLOT1 )
( MOVE R1 CRYO TUNNEL )
( MOVE R1 TUNNEL STASIS )
( UNLOAD R1 CS1 STASIS SLOT2 )
( UNLOAD R1 CS2 STASIS SLOT1 )
( MOVE R1 STASIS TUNNEL )
( MOVE R1 TUNNEL HALLB )
( LOAD R1 B1 HALLB SLOT2 )
( LOAD R1 B2 HALLB SLOT1 )
( MOVE R1 HALLB TUNNEL )
( MOVE R1 TUNNEL POD1 )
( UNLOAD R1 B1 POD1 SLOT2 )
( MOVE R1 POD1 TUNNEL )
( MOVE R1 TUNNEL POD2 )
( UNLOAD R1 B2 POD2 SLOT1 )
```

Fig4. FF-plan

```
(move r1 entrance tunnel)
(move r1 tunnel halla)
(load r1 a1 halla slot1)
(load r1 a2 halla slot2)
(move r1 halla tunnel)
(move r1 tunnel cryo)
(unload r1 a1 cryo slot1)
(load r1 cs1 cryo slot1)
(unload r1 a2 cryo slot2)
(load r1 cs2 cryo slot2)
(move r1 cryo tunnel)
(move r1 tunnel stasis)
(unload r1 cs1 stasis slot1)
(unload r1 cs2 stasis slot2)
(move r1 stasis tunnel)
(move r1 tunnel hallb)
(load r1 b1 hallb slot1)
(load r1 b2 hallb slot2)
(move r1 hallb tunnel)
(move r1 tunnel pod1)
(unload r1 b1 pod1 slot1)
(move r1 pod1 tunnel)
(move r1 tunnel pod2)
(unload r1 b2 pod2 slot2)
; cost = 24 (unit cost)
```

Fig5. LAMA-first-plan

```
(move entrance tunnel)
(move tunnel halla)
(load r1 a1 halla slot1)
(load r1 a2 halla slot2)
(move halla tunnel)
(move tunnel cryo)
(unload r1 a1 cryo slot1)
(load r1 cs1 cryo slot1)
(unload r1 a2 cryo slot2)
(load r1 cs2 cryo slot2)
(move cryo tunnel)
(move tunnel stasis)
(unload r1 cs1 stasis slot1)
(unload r1 cs2 stasis slot2)
(move stasis tunnel)
(move tunnel hallb)
(load r1 b1 hallb slot1)
(load r1 b2 hallb slot2)
(move hallb tunnel)
(move tunnel pod1)
(unload r1 b1 pod1 slot1)
(move pod1 tunnel)
(move tunnel pod2)
(unload r1 b2 pod2 slot2)
; cost = 24 (unit cost)
```

Fig6. LAMA.plan

**Observed result for the unassigned curator–drone model:**

- the saved FF-style(Fig7) curator–drone plans use only **d1** and ignore r1, leading to a **32-action** plan
- just adding a second agent does **not** automatically produce a better plan
- because curator and drone were modeled with almost the same capabilities, the planner may choose only one robot
- The best saved unassigned LAMA (Fig9)plan is **24 actions**, which is effectively as good as the single-curator two-slot case.

```
( MOVE D1 ENTRANCE TUNNEL )
( MOVE D1 TUNNEL HALLA )
( LOAD D1 A1 HALLA SLOT_D )
( MOVE D1 HALLA TUNNEL )
( MOVE D1 TUNNEL CRYO )
( UNLOAD D1 A1 CRYO SLOT_D )
( LOAD D1 CS1 CRYO SLOT_D )
( MOVE D1 CRYO TUNNEL )
( MOVE D1 TUNNEL STASIS )
( UNLOAD D1 CS1 STASIS SLOT_D )
( MOVE D1 STASIS TUNNEL )
( MOVE D1 TUNNEL HALLA )
( LOAD D1 A2 HALLA SLOT_D )
( MOVE D1 HALLA TUNNEL )
( MOVE D1 TUNNEL CRYO )
( UNLOAD D1 A2 CRYO SLOT_D )
( LOAD D1 CS2 CRYO SLOT_D )
( MOVE D1 CRYO TUNNEL )
( MOVE D1 TUNNEL STASIS )
( UNLOAD D1 CS2 STASIS SLOT_D )
( MOVE D1 STASIS TUNNEL )
( MOVE D1 TUNNEL HALLB )
( LOAD D1 B1 HALLB SLOT_D )
( MOVE D1 HALLB TUNNEL )
( MOVE D1 TUNNEL POD1 )
( UNLOAD D1 B1 POD1 SLOT_D )
( MOVE D1 POD1 TUNNEL )
( MOVE D1 TUNNEL HALLB )
( LOAD D1 B2 HALLB SLOT_D )
( MOVE D1 HALLB TUNNEL )
( MOVE D1 TUNNEL POD2 )
( UNLOAD D1 B2 POD2 SLOT_D )
```

Fig7. FF-curator–drone.plan

```
(move r1 entrance tunnel)
(move r1 tunnel cryo)
(load r1 cs1 cryo slot1)
(load r1 cs2 cryo slot2)
(move d1 entrance tunnel)
(move r1 cryo tunnel)
(move r1 tunnel stasis)
(unload r1 cs1 stasis slot1)
(unload r1 cs2 stasis slot2)
(move r1 stasis tunnel)
(move r1 tunnel hallb)
(load r1 b1 hallb slot1)
(load r1 b2 hallb slot2)
(move r1 hallb tunnel)
(move r1 tunnel pod1)
(unload r1 b1 pod1 slot1)
(move r1 pod1 tunnel)
(move r1 tunnel pod2)
(unload r1 b2 pod2 slot2)
(move r1 pod2 tunnel)
(move r1 tunnel halla)
(load r1 a1 halla slot1)
(load r1 a2 halla slot2)
(move r1 halla tunnel)
(move r1 tunnel cryo)
(unload r1 a1 cryo slot1)
(unload r1 a2 cryo slot2)
; cost = 27 (unit cost)
```

Fig8. LAMA-first-curator–drone-plan

```
(move r1 entrance tunnel)
(move r1 tunnel cryo)
(load r1 cs1 cryo slot1)
(load r1 cs2 cryo slot2)
(move d1 entrance tunnel)
(move r1 cryo tunnel)
(move r1 tunnel stasis)
(unload r1 cs1 stasis slot1)
(unload r1 cs2 stasis slot2)
(move d1 tunnel cryo)
(move r1 stasis tunnel)
(move r1 tunnel halla)
(load r1 a1 halla slot1)
(load r1 a2 halla slot2)
(move r1 halla tunnel)
(move r1 tunnel cryo)
(unload r1 a1 cryo slot1)
(unload r1 a2 cryo slot2)
(move r1 cryo tunnel)
(move r1 tunnel hallb)
(load r1 b1 hallb slot1)
(load r1 b2 hallb slot2)
(move r1 hallb tunnel)
(move r1 tunnel pod1)
(unload r1 b1 pod1 slot1)
(move r1 pod1 tunnel)
(move r1 tunnel pod2)
(unload r1 b2 pod2 slot2)
; cost = 28 (unit cost)
```

Fig9. LAMA-curator–drone.plan

**Observed result for the assigned model:**

- the assigned curator–drone plans use **both** `r1` and `d1`
- they have about **30 actions** (Fig10, 11 & 12)
- this is worse than the best 24-action unassigned plan in pure action count
- however, it demonstrates the intended division of labour more clearly
- in other words, the assigned model is better for showing cooperation, not for minimizing action count
- for shortest classical plan, the best Problem 2 result is the 24-action two-slot solution
- for showing multi-agent cooperation, the assigned domain is better, even though it is longer

```
1   ( MOVE R1 ENTRANCE TUNNEL )
2   ( MOVE D1 ENTRANCE TUNNEL )
3   ( MOVE R1 TUNNEL HALLA )
4   ( LOAD R1 A1 HALLA SLOT2 )
5   ( LOAD R1 A2 HALLA SLOT1 )
6   ( MOVE R1 HALLA TUNNEL )
7   ( MOVE R1 TUNNEL CRYO )
8   ( UNLOAD R1 A1 CRYO SLOT2 )
9   ( UNLOAD R1 A2 CRYO SLOT1 )
10  ( MOVE R1 CRYO TUNNEL )
11  ( MOVE R1 TUNNEL HALLB )
12  ( LOAD R1 B1 HALLB SLOT2 )
13  ( LOAD R1 B2 HALLB SLOT1 )
14  ( MOVE R1 HALLB TUNNEL )
15  ( MOVE R1 TUNNEL POD1 )
16  ( UNLOAD R1 B1 POD1 SLOT2 )
17  ( MOVE R1 POD1 TUNNEL )
18  ( MOVE R1 TUNNEL POD2 )
19  ( UNLOAD R1 B2 POD2 SLOT1 )
20  ( MOVE D1 TUNNEL CRYO )
21  ( LOAD D1 CS1 CRYO SLOT_D )
22  ( MOVE D1 CRYO TUNNEL )
23  ( MOVE D1 TUNNEL STASIS )
24  ( UNLOAD D1 CS1 STASIS SLOT_D )
25  ( MOVE D1 STASIS TUNNEL )
26  ( MOVE D1 TUNNEL CRYO )
27  ( LOAD D1 CS2 CRYO SLOT_D )
28  ( MOVE D1 CRYO TUNNEL )
29  ( MOVE D1 TUNNEL STASIS )
30  ( UNLOAD D1 CS2 STASIS SLOT_D )
```

Fig10. FF-curator-drone-assigned

```
1   (move r1 entrance tunnel)
2   (move d1 entrance tunnel)
3   (move r1 tunnel halla)
4   (load r1 a1 halla slot1)
5   (load r1 a2 halla slot2)
6   (move r1 halla tunnel)
7   (move r1 tunnel cryo)
8   (unload r1 a1 cryo slot1)
9   (unload r1 a2 cryo slot2)
10  (move r1 cryo tunnel)
11  (move r1 tunnel hallb)
12  (load r1 b1 hallb slot1)
13  (load r1 b2 hallb slot2)
14  (move r1 hallb tunnel)
15  (move r1 tunnel pod1)
16  (unload r1 b1 pod1 slot1)
17  (move r1 pod1 tunnel)
18  (move r1 tunnel pod2)
19  (unload r1 b2 pod2 slot2)
20  (move d1 tunnel cryo)
21  (load d1 cs1 cryo slot_d)
22  (move d1 cryo tunnel)
23  (move d1 tunnel stasis)
24  (unload d1 cs1 stasis slot_d)
25  (move d1 stasis tunnel)
26  (move d1 tunnel cryo)
27  (load d1 cs2 cryo slot_d)
28  (move d1 cryo tunnel)
29  (move d1 tunnel stasis)
30  (unload d1 cs2 stasis slot_d)
31  ; cost = 30 (unit cost)
```

Fig11. LAMA-FIRST-curator-drone-assigned

```
1   (move r1 entrance tunnel)
2   (move d1 entrance tunnel)
3   (move r1 tunnel halla)
4   (load r1 a1 halla slot1)
5   (load r1 a2 halla slot2)
6   (move r1 halla tunnel)
7   (move r1 tunnel cryo)
8   (unload r1 a1 cryo slot1)
9   (unload r1 a2 cryo slot2)
10  (move r1 cryo tunnel)
11  (move r1 tunnel hallb)
12  (load r1 b1 hallb slot1)
13  (load r1 b2 hallb slot2)
14  (move r1 hallb tunnel)
15  (move r1 tunnel pod1)
16  (unload r1 b1 pod1 slot1)
17  (move r1 pod1 tunnel)
18  (move r1 tunnel pod2)
19  (unload r1 b2 pod2 slot2)
20  (move d1 tunnel cryo)
21  (load d1 cs1 cryo slot_d)
22  (move d1 cryo tunnel)
23  (move d1 tunnel stasis)
24  (unload d1 cs1 stasis slot_d)
25  (move d1 stasis tunnel)
26  (move d1 tunnel cryo)
27  (load d1 cs2 cryo slot_d)
28  (move d1 cryo tunnel)
29  (move d1 tunnel stasis)
30  (unload d1 cs2 stasis slot_d)
31  ; cost = 30 (unit cost)
```

Fig12. LAMA-curator-drone-assigned

## 6.3 Problem 3: HTN / HDDL

**Observed result:**

- The main HTN model handles each delivery more independently and often returns the robot to a common restart point with about **46 primitive actions** (Fig13)**.**
- The paired HTN version groups deliveries and uses the two-slot capacity more effectively
- The **paired HTN variant** is better as a plan with about **30 primitive actions** (Fig14)**.**
- it reduces unnecessary travel and is much closer to the more efficient classical Problem 2 capacity behavior

```
SOLUTION SEQUENCE
0: move(r1,entrance,tunnel)
1: move(r1,tunnel,hallA)
2: load(r1,a1,hallA,slot1)
3: move(r1,hallA,tunnel)
4: move(r1,tunnel,cryo)
5: unload(r1,a1,cryo,slot1)
6: move(r1,cryo,tunnel)
7: move(r1,tunnel,entrance)
8: move(r1,entrance,tunnel)
9: move(r1,tunnel,hallA)
10: load(r1,a2,hallA,slot2)
11: move(r1,hallA,tunnel)
12: move(r1,tunnel,cryo)
13: unload(r1,a2,cryo,slot2)
14: move(r1,cryo,tunnel)
15: move(r1,tunnel,entrance)
16: move(r1,entrance,tunnel)
17: move(r1,tunnel,hallB)
18: load(r1,b1,hallB,slot1)
19: move(r1,hallB,tunnel)
20: move(r1,tunnel,pod1)
21: unload(r1,b1,pod1,slot1)
22: move(r1,pod1,tunnel)
23: move(r1,tunnel,entrance)
24: move(r1,entrance,tunnel)
25: move(r1,tunnel,hallB)
26: load(r1,b2,hallB,slot2)
27: move(r1,hallB,tunnel)
28: move(r1,tunnel,pod2)
29: unload(r1,b2,pod2,slot2)
30: move(r1,pod2,tunnel)
31: move(r1,tunnel,entrance)
32: move(r1,entrance,tunnel)
33: move(r1,tunnel,cryo)
34: load(r1,cs1,cryo,slot1)
35: move(r1,cryo,tunnel)
36: move(r1,tunnel,stasis)
37: unload(r1,cs1,stasis,slot1)
38: move(r1,stasis,tunnel)
39: move(r1,tunnel,entrance)
40: move(r1,entrance,tunnel)
41: move(r1,tunnel,cryo)
42: load(r1,cs2,cryo,slot2)
43: move(r1,cryo,tunnel)
44: move(r1,tunnel,stasis)
45: unload(r1,cs2,stasis,slot2)
```

Fig13. panda.plan

```
SOLUTION SEQUENCE
0: move(r1,entrance,tunnel)
1: move(r1,tunnel,hallA)
2: load(r1,a1,hallA,slot1)
3: load(r1,a2,hallA,slot2)
4: move(r1,hallA,tunnel)
5: move(r1,tunnel,cryo)
6: unload(r1,a1,cryo,slot1)
7: unload(r1,a2,cryo,slot2)
8: move(r1,cryo,tunnel)
9: move(r1,tunnel,entrance)
10: move(r1,entrance,tunnel)
11: move(r1,tunnel,hallB)
12: load(r1,b1,hallB,slot1)
13: load(r1,b2,hallB,slot2)
14: move(r1,hallB,tunnel)
15: move(r1,tunnel,pod1)
16: unload(r1,b1,pod1,slot1)
17: move(r1,pod1,tunnel)
18: move(r1,tunnel,pod2)
19: unload(r1,b2,pod2,slot2)
20: move(r1,pod2,tunnel)
21: move(r1,tunnel,entrance)
22: move(r1,entrance,tunnel)
23: move(r1,tunnel,cryo)
24: load(r1,cs1,cryo,slot1)
25: load(r1,cs2,cryo,slot2)
26: move(r1,cryo,tunnel)
27: move(r1,tunnel,stasis)
28: unload(r1,cs1,stasis,slot1)
29: unload(r1,cs2,stasis,slot2)
```

Fig14. panda-pair.plan

## 6.4 Problem 4: temporal / seismic planning

**Observed result for the single-robot seismic plan:**

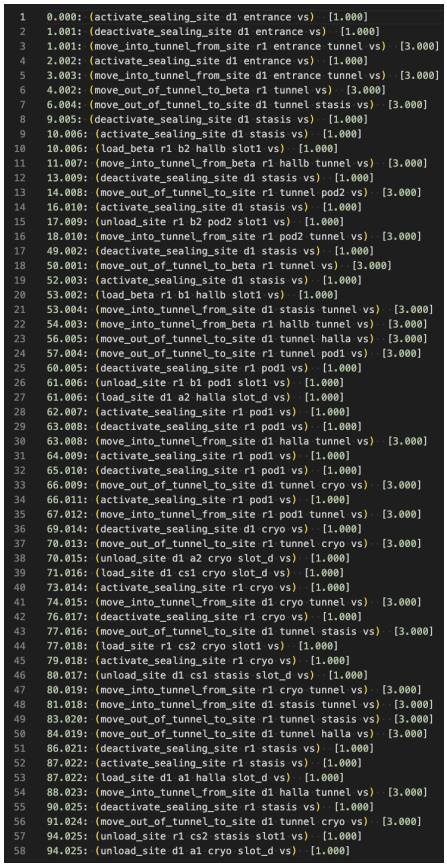
- `optic-seismic-raw.plan` and `optic-seismic-val.plan` contain the **same logical plan,** but the `-val` version only shifts timestamps slightly so that VAL accepts the schedule more reliably
- the single-robot temporal plan has about **50 durative actions** (Fig15).

```
0.000: (activate_sealing_site r1 entrance vs) [1.000]
1.020: (move_into_tunnel_from_site r1 entrance tunnel vs) [3.000]
4.030: (move_out_of_tunnel_to_site r1 tunnel halla vs) [3.000]
7.040: (deactivate_sealing_site r1 halla vs) [1.000]
8.060: (load_site r1 a2 halla slot1 vs) [1.000]
9.070: (activate_sealing_site r1 halla vs) [1.000]
10.090: (move_into_tunnel_from_site r1 halla tunnel vs) [3.000]
13.100: (move_out_of_tunnel_to_site r1 tunnel cryo vs) [3.000]
16.110: (deactivate_sealing_site r1 cryo vs) [1.000]
17.130: (unload_site r1 a2 cryo slot1 vs) [1.000]
18.140: (load_site r1 cs2 cryo slot1 vs) [1.000]
19.150: (activate_sealing_site r1 cryo vs) [1.000]
20.170: (move_into_tunnel_from_site r1 cryo tunnel vs) [3.000]
23.180: (move_out_of_tunnel_to_site r1 tunnel stasis vs) [3.000]
26.190: (deactivate_sealing_site r1 stasis vs) [1.000]
27.210: (unload_site r1 cs2 stasis slot1 vs) [1.000]
28.220: (activate_sealing_site r1 stasis vs) [1.000]
29.240: (move_into_tunnel_from_site r1 stasis tunnel vs) [3.000]
32.250: (move_out_of_tunnel_to_site r1 tunnel halla vs) [3.000]
35.260: (deactivate_sealing_site r1 halla vs) [1.000]
36.280: (load_site r1 a1 halla slot1 vs) [1.000]
37.290: (activate_sealing_site r1 halla vs) [1.000]
38.310: (move_into_tunnel_from_site r1 halla tunnel vs) [3.000]
41.320: (move_out_of_tunnel_to_site r1 tunnel cryo vs) [3.000]
44.330: (deactivate_sealing_site r1 cryo vs) [1.000]
45.350: (unload_site r1 a1 cryo slot1 vs) [1.000]
46.360: (load_site r1 cs1 cryo slot1 vs) [1.000]
47.370: (activate_sealing_site r1 cryo vs) [1.000]
48.390: (move_into_tunnel_from_site r1 cryo tunnel vs) [3.000]
51.400: (move_out_of_tunnel_to_site r1 tunnel stasis vs) [3.000]
54.410: (deactivate_sealing_site r1 stasis vs) [1.000]
55.430: (unload_site r1 cs1 stasis slot1 vs) [1.000]
56.440: (activate_sealing_site r1 stasis vs) [1.000]
57.460: (move_into_tunnel_from_site r1 stasis tunnel vs) [3.000]
60.470: (move_out_of_tunnel_to_site r1 tunnel pod2 vs) [3.000]
63.480: (move_into_tunnel_from_site r1 pod2 tunnel vs) [3.000]
66.490: (move_out_of_tunnel_to_beta r1 tunnel vs) [3.000]
69.500: (deactivate_sealing_beta r1 hallb vs) [1.000]
70.520: (load_beta r1 b2 hallb slot1 vs) [1.000]
71.530: (load_beta r1 b1 hallb slot2 vs) [1.000]
72.540: (activate_sealing_beta r1 hallb vs) [1.000]
73.560: (move_into_tunnel_from_beta r1 hallb tunnel vs) [3.000]
76.570: (move_out_of_tunnel_to_site r1 tunnel pod1 vs) [3.000]
79.580: (deactivate_sealing_site r1 pod1 vs) [1.000]
80.600: (unload_site r1 b1 pod1 slot2 vs) [1.000]
81.610: (activate_sealing_site r1 pod1 vs) [1.000]
82.630: (move_into_tunnel_from_site r1 pod1 tunnel vs) [3.000]
85.640: (move_out_of_tunnel_to_site r1 tunnel pod2 vs) [3.000]
88.650: (deactivate_sealing_site r1 pod2 vs) [1.000]
89.670: (unload_site r1 b2 pod2 slot1 vs) [1.000]
```

Fig15. optic-seismic-val.plan

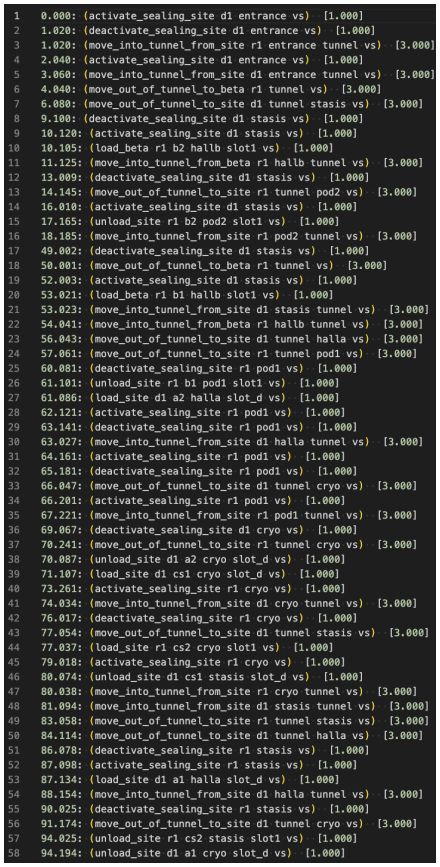
**Observed result for the curator–drone seismic plan:**

- The drone plan contains about **58 durative actions** (Fig16 & 17)**.** It clearly shows temporal overlap between `r1` and `d1`.However, the total schedule is not automatically shorter than the single-robot version.
- temporal parallelism exists, but shared sealing and Hall Beta timing create overhead.
- So, the two-agent temporal plan is better for demonstrating concurrency, not necessarily for minimizing makespan in this specific domain.
- **OPTIC** is the most useful planner here because it handles the seismic temporal model directly.
- The **single-robot seismic plan** is cleaner and easier to validate.
- The **drone plan** is better as evidence of parallelism, even if it is not always shorter.



```
0.000: (activate_sealing_site d1 entrance vs)  [1.000]
1.001: (deactivate_sealing_site d1 entrance vs)  [1.000]
1.001: (move_into_tunnel_from_site r1 entrance tunnel vs)  [3.000]
2.002: (activate_sealing_site d1 entrance vs)  [1.000]
3.003: (move_into_tunnel_from_site d1 entrance tunnel vs)  [3.000]
4.002: (move_out_of_tunnel_to_beta r1 tunnel vs)  [3.000]
6.004: (move_out_of_tunnel_to_site d1 tunnel stasis vs)  [3.000]
9.005: (deactivate_sealing_site d1 stasis vs)  [1.000]
10.006: (activate_sealing_site d1 stasis vs)  [1.000]
10.006: (load_beta r1 b2 hallb slot1 vs)  [1.000]
11.007: (move_into_tunnel_from_beta r1 hallb tunnel vs)  [3.000]
13.009: (deactivate_sealing_site d1 stasis vs)  [1.000]
14.008: (move_out_of_tunnel_to_site r1 tunnel pod2 vs)  [3.000]
16.010: (activate_sealing_site d1 stasis vs)  [1.000]
17.009: (unload_site r1 b2 pod2 slot1 vs)  [1.000]
18.010: (move_into_tunnel_from_site r1 pod2 tunnel vs)  [3.000]
49.002: (deactivate_sealing_site d1 stasis vs)  [1.000]
50.001: (move_out_of_tunnel_to_beta r1 tunnel vs)  [3.000]
52.003: (activate_sealing_site d1 stasis vs)  [1.000]
53.002: (load_beta r1 b1 hallb slot1 vs)  [1.000]
53.004: (move_into_tunnel_from_site d1 stasis tunnel vs)  [3.000]
54.003: (move_into_tunnel_from_beta r1 hallb tunnel vs)  [3.000]
56.005: (move_out_of_tunnel_to_site d1 tunnel halla vs)  [3.000]
57.004: (move_out_of_tunnel_to_site r1 tunnel pod1 vs)  [3.000]
60.005: (deactivate_sealing_site r1 pod1 vs)  [1.000]
61.006: (unload_site r1 b1 pod1 slot1 vs)  [1.000]
61.006: (load_site d1 a2 halla slot_d vs)  [1.000]
62.007: (activate_sealing_site r1 pod1 vs)  [1.000]
63.008: (deactivate_sealing_site r1 pod1 vs)  [1.000]
63.008: (move_into_tunnel_from_site d1 halla tunnel vs)  [3.000]
64.009: (activate_sealing_site r1 pod1 vs)  [1.000]
65.010: (deactivate_sealing_site r1 pod1 vs)  [1.000]
66.009: (move_out_of_tunnel_to_site d1 tunnel cryo vs)  [3.000]
66.011: (activate_sealing_site r1 pod1 vs)  [1.000]
67.012: (move_into_tunnel_from_site r1 pod1 tunnel vs)  [3.000]
69.014: (deactivate_sealing_site d1 cryo vs)  [1.000]
70.013: (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3.000]
70.015: (unload_site d1 a2 cryo slot_d vs)  [1.000]
71.016: (load_site d1 cs1 cryo slot_d vs)  [1.000]
73.014: (activate_sealing_site r1 cryo vs)  [1.000]
74.015: (move_into_tunnel_from_site d1 cryo tunnel vs)  [3.000]
76.017: (deactivate_sealing_site r1 cryo vs)  [1.000]
77.016: (move_out_of_tunnel_to_site d1 tunnel stasis vs)  [3.000]
77.018: (load_site r1 cs2 cryo slot1 vs)  [1.000]
79.018: (activate_sealing_site r1 cryo vs)  [1.000]
80.017: (unload_site d1 cs1 stasis slot_d vs)  [1.000]
80.019: (move_into_tunnel_from_site r1 cryo tunnel vs)  [3.000]
81.018: (move_into_tunnel_from_site d1 stasis tunnel vs)  [3.000]
83.020: (move_out_of_tunnel_to_site r1 tunnel stasis vs)  [3.000]
84.019: (move_out_of_tunnel_to_site d1 tunnel halla vs)  [3.000]
86.021: (deactivate_sealing_site r1 stasis vs)  [1.000]
87.022: (activate_sealing_site r1 stasis vs)  [1.000]
87.022: (load_site d1 a1 halla slot_d vs)  [1.000]
88.023: (move_into_tunnel_from_site d1 halla tunnel vs)  [3.000]
90.025: (deactivate_sealing_site r1 stasis vs)  [1.000]
91.024: (move_out_of_tunnel_to_site d1 tunnel cryo vs)  [3.000]
94.025: (unload_site r1 cs2 stasis slot1 vs)  [1.000]
94.025: (unload_site d1 a1 cryo slot_d vs)  [1.000]
```

Fig16. optic-seismic-drone.plan



```
0.000: (activate_sealing_site d1 entrance vs)  [1.000]
1.020: (deactivate_sealing_site d1 entrance vs)  [1.000]
1.020: (move_into_tunnel_from_site r1 entrance tunnel vs)  [3.000]
2.040: (activate_sealing_site d1 entrance vs)  [1.000]
3.060: (move_into_tunnel_from_site d1 entrance tunnel vs)  [3.000]
4.040: (move_out_of_tunnel_to_beta r1 tunnel vs)  [3.000]
6.080: (move_out_of_tunnel_to_site d1 tunnel stasis vs)  [3.000]
9.100: (deactivate_sealing_site d1 stasis vs)  [1.000]
10.120: (activate_sealing_site d1 stasis vs)  [1.000]
10.105: (load_beta r1 b2 hallb slot1 vs)  [1.000]
11.125: (move_into_tunnel_from_beta r1 hallb tunnel vs)  [3.000]
13.009: (deactivate_sealing_site d1 stasis vs)  [1.000]
14.145: (move_out_of_tunnel_to_site r1 tunnel pod2 vs)  [3.000]
16.010: (activate_sealing_site d1 stasis vs)  [1.000]
17.165: (unload_site r1 b2 pod2 slot1 vs)  [1.000]
18.185: (move_into_tunnel_from_site r1 pod2 tunnel vs)  [3.000]
49.002: (deactivate_sealing_site d1 stasis vs)  [1.000]
50.001: (move_out_of_tunnel_to_beta r1 tunnel vs)  [3.000]
52.003: (activate_sealing_site d1 stasis vs)  [1.000]
53.021: (load_beta r1 b1 hallb slot1 vs)  [1.000]
53.023: (move_into_tunnel_from_site d1 stasis tunnel vs)  [3.000]
54.041: (move_into_tunnel_from_beta r1 hallb tunnel vs)  [3.000]
56.043: (move_out_of_tunnel_to_site d1 tunnel halla vs)  [3.000]
57.061: (move_out_of_tunnel_to_site r1 tunnel pod1 vs)  [3.000]
60.081: (deactivate_sealing_site r1 pod1 vs)  [1.000]
61.101: (unload_site r1 b1 pod1 slot1 vs)  [1.000]
61.086: (load_site d1 a2 halla slot_d vs)  [1.000]
62.121: (activate_sealing_site r1 pod1 vs)  [1.000]
63.141: (deactivate_sealing_site r1 pod1 vs)  [1.000]
63.027: (move_into_tunnel_from_site d1 halla tunnel vs)  [3.000]
64.161: (activate_sealing_site r1 pod1 vs)  [1.000]
65.181: (deactivate_sealing_site r1 pod1 vs)  [1.000]
66.047: (move_out_of_tunnel_to_site d1 tunnel cryo vs)  [3.000]
66.201: (activate_sealing_site r1 pod1 vs)  [1.000]
67.221: (move_into_tunnel_from_site r1 pod1 tunnel vs)  [3.000]
69.067: (deactivate_sealing_site d1 cryo vs)  [1.000]
70.241: (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3.000]
70.087: (unload_site d1 a2 cryo slot_d vs)  [1.000]
71.107: (load_site d1 cs1 cryo slot_d vs)  [1.000]
73.261: (activate_sealing_site r1 cryo vs)  [1.000]
74.034: (move_into_tunnel_from_site d1 cryo tunnel vs)  [3.000]
76.017: (deactivate_sealing_site r1 cryo vs)  [1.000]
77.054: (move_out_of_tunnel_to_site d1 tunnel stasis vs)  [3.000]
77.037: (load_site r1 cs2 cryo slot1 vs)  [1.000]
79.018: (activate_sealing_site r1 cryo vs)  [1.000]
80.074: (unload_site d1 cs1 stasis slot_d vs)  [1.000]
80.038: (move_into_tunnel_from_site r1 cryo tunnel vs)  [3.000]
81.094: (move_into_tunnel_from_site d1 stasis tunnel vs)  [3.000]
83.058: (move_out_of_tunnel_to_site r1 tunnel stasis vs)  [3.000]
84.114: (move_out_of_tunnel_to_site d1 tunnel halla vs)  [3.000]
86.078: (deactivate_sealing_site r1 stasis vs)  [1.000]
87.098: (activate_sealing_site r1 stasis vs)  [1.000]
87.134: (load_site d1 a1 halla slot_d vs)  [1.000]
88.154: (move_into_tunnel_from_site d1 halla tunnel vs)  [3.000]
90.025: (deactivate_sealing_site r1 stasis vs)  [1.000]
91.174: (move_out_of_tunnel_to_site d1 tunnel cryo vs)  [3.000]
94.025: (unload_site r1 cs2 stasis slot1 vs)  [1.000]
94.194: (unload_site d1 a1 cryo slot_d vs)  [1.000]
```

Fig17. optic-seismic-drone-val.plan

## 6.5 Problem 5: PlanSys2 execution

**Observed result:**

- the full mission was executed through staged cumulative goals
- An earlier long final sequence was unreliable in PlanSys2, so splitting the mission into cumulative stages made the execution stable, where each stage ended with `Plan Succeeded.`
- the stages were:
  - Phase 1: move b1 to pod1 and move b2 to pod2 (Fig18)
  - Phase 2: move cs1 to stasis and move cs2 to stasis (Fig19)
  - Phase 3: move a1 to cryo and move a2 to cryo (Fig20)

```
> set goal (and (artifact_at b1 pod1))
> get plan
plan:
0:     (activate_sealing_site r1 entrance vs)  [1]
1.001:  (move_into_tunnel_from_site r1 entrance tunnel vs)      [3]
4.002:  (move_out_of_tunnel_to_beta r1 tunnel hallb vs) [3]
7.003:  (deactivate_sealing_beta r1 hallb vs)   [1]
8.004:  (load_beta r1 b1 hallb slot1 vs)        [1]
9.005:  (activate_sealing_beta r1 hallb vs)     [1]
10.006: (move_into_tunnel_from_beta r1 hallb tunnel vs) [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel pod1 vs)  [3]
16.008: (deactivate_sealing_site r1 pod1 vs)    [1]
17.009: (unload_site r1 b1 pod1 slot1 vs)       [1]
> run
[INFO] [1777929221.004431006] [executor_client]: Plan Succeeded

Successful finished
> set goal (and (artifact_at b1 pod1) (artifact_at b2 pod2))
> get plan
plan:
0:     (activate_sealing_site r1 pod1 vs)       [1]
1.001:  (move_into_tunnel_from_site r1 pod1 tunnel vs)  [3]
4.002:  (move_out_of_tunnel_to_beta r1 tunnel hallb vs) [3]
7.003:  (deactivate_sealing_beta r1 hallb vs)   [1]
8.004:  (load_beta r1 b2 hallb slot1 vs)        [1]
9.005:  (activate_sealing_beta r1 hallb vs)     [1]
10.006: (move_into_tunnel_from_beta r1 hallb tunnel vs) [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel pod2 vs)  [3]
16.008: (deactivate_sealing_site r1 pod2 vs)    [1]
17.009: (unload_site r1 b2 pod2 slot1 vs)       [1]
> run
[INFO] [1777929309.991004422] [executor_client]: Plan Succeeded
```

Fig18. Phase1

```
> set goal (and (artifact_at b1 pod1) (artifact_at b2 pod2) (artifact_at cs1 stasis))
> get plan
plan:
0:     (activate_sealing_site r1 pod2 vs)       [1]
1.001:  (move_into_tunnel_from_site r1 pod2 tunnel vs)  [3]
4.002:  (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3]
7.003:  (deactivate_sealing_site r1 cryo vs)    [1]
8.004:  (load_site r1 cs1 cryo slot1 vs)        [1]
9.005:  (activate_sealing_site r1 cryo vs)      [1]
10.006: (move_into_tunnel_from_site r1 cryo tunnel vs)  [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel stasis vs)        [3]
16.008: (deactivate_sealing_site r1 stasis vs)  [1]
17.009: (unload_site r1 cs1 stasis slot1 vs)    [1]
> run
[INFO] [1777929391.237853543] [executor_client]: Plan Succeeded

Successful finished
> set goal (and (artifact_at b1 pod1) (artifact_at b2 pod2) (artifact_at cs1 stasis) (artifact_at cs2 s
> get plan
plan:
0:     (activate_sealing_site r1 stasis vs)     [1]
1.001:  (move_into_tunnel_from_site r1 stasis tunnel vs)        [3]
4.002:  (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3]
7.003:  (deactivate_sealing_site r1 cryo vs)    [1]
8.004:  (load_site r1 cs2 cryo slot1 vs)        [1]
9.005:  (activate_sealing_site r1 cryo vs)      [1]
10.006: (move_into_tunnel_from_site r1 cryo tunnel vs)  [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel stasis vs)        [3]
16.008: (deactivate_sealing_site r1 stasis vs)  [1]
17.009: (unload_site r1 cs2 stasis slot1 vs)    [1]
> run
[INFO] [1777929469.301371552] [executor_client]: Plan Succeeded

Successful finished
```

Fig19. Phase2

```
> set goal (and (artifact_at b1 pod1) (artifact_at b2 pod2) (artifact_at cs1 stasis)
            (artifact_at cs2 stasis) (artifact_at a1 cryo))
> get plan
plan:
0:     (activate_sealing_site r1 stasis vs)     [1]
1.001:  (move_into_tunnel_from_site r1 stasis tunnel vs)        [3]
4.002:  (move_out_of_tunnel_to_site r1 tunnel halla vs) [3]
7.003:  (deactivate_sealing_site r1 halla vs)   [1]
8.004:  (load_site r1 a1 halla slot1 vs)        [1]
9.005:  (activate_sealing_site r1 halla vs)     [1]
10.006: (move_into_tunnel_from_site r1 halla tunnel vs) [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3]
16.008: (deactivate_sealing_site r1 cryo vs)    [1]
17.009: (unload_site r1 a1 cryo slot1 vs)       [1]
> run
[INFO] [1777929547.960255838] [executor_client]: Plan Succeeded

Successful finished
> set goal (and (artifact_at a1 cryo) (artifact_at a2 cryo) (artifact_at b1 pod1)
            (artifact_at b2 pod2) (artifact_at cs1 stasis) (artifact_at cs2 stasis))
> get plan
plan:
0:     (activate_sealing_site r1 cryo vs)       [1]
1.001:  (move_into_tunnel_from_site r1 cryo tunnel vs)  [3]
4.002:  (move_out_of_tunnel_to_site r1 tunnel halla vs) [3]
7.003:  (deactivate_sealing_site r1 halla vs)   [1]
8.004:  (load_site r1 a2 halla slot1 vs)        [1]
9.005:  (activate_sealing_site r1 halla vs)     [1]
10.006: (move_into_tunnel_from_site r1 halla tunnel vs) [3]
13.007: (move_out_of_tunnel_to_site r1 tunnel cryo vs)  [3]
16.008: (deactivate_sealing_site r1 cryo vs)    [1]
17.009: (unload_site r1 a2 cryo slot1 vs)       [1]
> run
[INFO] [1777929624.150778304] [executor_client]: Plan Succeeded

Successful finished
```

Fig20. Phase3

# 7. References

[1] J. Benton, Amanda Jane Coles, and Andrew Coles.Temporal planning with preferences and time-dependent continuous costs. In Lee McCluskey, Brian Charles Williams, Jos´ e Reinaldo Silva, and Blai Bonet, editors, Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling, ICAPS 2012, Atibaia, S˜ ao Paulo, Brazil, June 25-19, 2012. AAAI, 2012.
[2] Malte Helmert. The fast downward planning system. J. Artif. Intell. Res., 26:191–246, 2006.
[3] Francisco Mart´ ın and colleagues. PlanSys2 Tutorials, 2022.
[4] Francisco Mart´ ın, Jonatan Gin´ es, Francisco J.Rodr´ ıguez, and Vicente Matell´ an. PlanSys2: A Planning System Framework for ROS2. In IEEE/RSJInternational Conference on Intelligent Robots and Systems, IROS 2021, Prague, Czech Republic, September 27 - October 1, 2021. IEEE, 2021.
[5] Cristian Muise and other contributors. PLANUTILS, 2021. General library for setting up linux-based environments for developing, running,and evaluating planners.